

Peephole connections (Gers and Schmidhuber, 2000) provide gates with direct read access to the cell state

A Bidirectional LSTM (Schuster et al., 1997) has advantages over the unidirectional version if the input sequences do not have a preferred processing direction.

Convolutional LSTM (Shi et al., 2015) can process sequences of images, e.g., videos.

Transformer networks use multi-head attention, which works as follows. We run multiple instances of scaled dot-product attention, concatenate their outputs (or context) and feed the result into a linear layer.

Without positional embeddings, the Transformer network has no way of figuring out the order of elements in an input sequence.

Sequence-to-sequence learning tasks without one-to-one correspondences can be tackled using an encoder-decoder architecture.

Alex Graves showed in his 2013 paper how LSTM can be used to generate text sequences simply by predicting one character at a time.

Fan et al. (2015) trained an LSTM to predict facial features from audio/text inputs. On this basis, given a speech recording, they could generate photo-realistic videos of talking heads.

LSTM can be used for protein classification. Hereby, the protein is represented as a sequence of amino acids and the labels are either functional or structural properties of the respective protein.

The so-called "update gate" of the GRU acts as a coupled input and forget gate

A forget gate mechanism is the core of the GRU architecture.

In the GRU, a so-called "reset gate" $r(t)$ regulates how strongly the increments $z(t)$ depend on the hidden states $h(t)$.

An LSTM with self-attention has access to all previous hidden states and cell states via an softmax-based attention mechanism.

The LSTM gates can be interpreted as a temporal attention mechanism.

RTRL is very slow and therefore rarely used in practice.

Truncated BPTT divides the input sequence into chunks and propagates the deltas only within chunks but not across them.

RTRL computes the gradients $\partial L(t)/\partial \theta$ during the forward pass already.

An LSTM with coupled input gate and forget gate, i.e., $c(t) = (1 - i(t)) \odot c(t - 1) + i(t) \odot z(t)$, leads (alongside other changes) to the gated recurrent unit (GRU).

"Ticker steps" refers to the technique of letting an LSTM run for more time steps even after the input sequence has been taken in by the network.

Most LSTM networks used in practice feature a forget gate.

The focused LSTM network feeds the input vector $x(t)$ only into the block input $z(t)$ and not into input gate $i(t)$ or output gate $o(t)$

If we choose $z(t) = \sigma(Ux(t) + Ph(t-1))$, then $E[z(t)] > 0$ and the memory cells will always increase at every time step. This can be a problem for very long sequences but can also be helpful for certain problems.

The memory cell fulfills $\partial c(t)/\partial c(t-r) = I$ for any $r \in \{1, \dots, t-1\}$, where I is the identity matrix. This solves the vanishing gradient problem and is called constant error carousel.

Elman proposed to train this network locally in time, i.e., on each time step separately.

In Elman The recurrent weights are constant.

In Elman The recurrent weights are equal to the identity matrix

RNNs can be applied to input sequences of varying lengths.

An RNN is equivalent to a feedforward network applied to the input and an internal state representation that depends on previous time steps.

LSTM needs special dropout techniques because a naive application will reset any memory cell with exponentially high probability.

Dropout is a regularization technique that randomly sets activations to zero during the forward pass.

The central idea of zoneout is to randomly clear the updates to the memory cell instead of its content.

Part-of-speech tagging is an example for sequence-to-sequence learning with one-to-one correspondences.

The forget gate should be initialized with positive bias values.

Karpathy and Fei-Fei (2015) trained two networks, a CNN and an RNN, for the task of image captioning.

In the Transformer Decoder, the attention matrix is designed with a masking mechanism, ensuring that each token in the input sequence attends only to preceding tokens in the sequence.

The vector at position t in the output sequence of a self-attention layer is computed by summing the value vectors, each weighted by its corresponding attention score from the t -th row of the attention matrix.

Although the attention matrix is constant for all input sequences, it undergoes gradual updates during training to minimize the loss function - **False, it is not constant and is computed for each input sequence based on Q, K, V. Second, the attention matrix is not updated, rather model's parameters (including those that generate Q, K, V) are updated**

The Transformer architecture applies convolutional layers between each self-attention layer to better capture local dependencies in the input data. **false, it applies feed-forward layers**

Multi-head Attention is designed such that each attention head is forced to attend to a different part of the hidden space. **false, the model learns to attend to different parts on its own**

In the standard self-attention formulation, cosine similarity is used as a measure of alignment between keys and values. **false, dot-product similarity is used.**

Similar to the hidden state in an LSTM, the attention matrix is updated in a recurrent manner for each token in an input sequence. **false, attention for all tokens in input is computed in parallel, which is a key distinction from LSTM.**

